Large Language Models (LLMs)

6 pages • Transformers, tokenization, prompting, generation, fine-tuning and applications

1. LLM Fundamentals

Definition

A Large Language Model (LLM) is a neural network trained on large text datasets to model language and generate or interpret text. Most modern LLMs use the Transformer architecture.

Capabilities

Question answering, summarization, rewriting, code generation, translation, classification, extraction and structured output.

Tokens

Text is split into tokens—word pieces, characters or other units. Token counts are not equal to word counts.

Context window

The maximum number of input and generated tokens a model can process in one request.

Parameters

Learned numerical values inside the model. More parameters do not automatically guarantee better task performance.

Pretraining

Learns general language patterns, commonly by predicting the next token from preceding context.

Post-training

Improves instruction following, helpfulness and safety using supervised examples and preference-based methods.

Important limitation

An LLM predicts likely continuations from context. It does not automatically verify that every generated statement is true.

Key takeaway: LLM applications are systems: prompts, retrieval, tools, validation and monitoring all affect final quality.

2. Transformer Architecture

Why Transformers?

Attention lets tokens relate to other tokens across a sequence. Transformers support parallel processing during training and model long-range dependencies.

Tokenization and embeddings

A tokenizer converts text to token IDs. An embedding layer maps each ID to a dense vector representation.

Positional information

Position encodings or rotary position embeddings provide information about token order.

Self-attention

Each token representation is projected into Query (Q), Key (K) and Value (V). Queries compare with keys to determine how values should be combined.

Scaled dot-product attention

Attention(Q,K,V) = softmax(QK■ / √d■)V.

Multi-head attention

Several attention heads learn different relationships in parallel; their outputs are combined and projected.

Feed-forward network

Transforms each token representation after attention.

Residual connections and normalization

Help information flow through layers and support stable training.

Causal masking

In decoder-only generative models, a token can attend to earlier tokens but not future tokens while predicting the next token.

Key takeaway: Common architecture flow: token IDs → embeddings + position information → repeated attention/FFN blocks → output scores.

3. LLM Text Generation & Decoding

Autoregressive generation

The model predicts a distribution for the next token, selects one, appends it to context and repeats until a stop condition.

Greedy decoding

Always chooses the highest-probability next token; often predictable but may be repetitive.

Temperature

Controls distribution sharpness. Lower values are more deterministic; higher values increase variation.

Top-k sampling

Samples only from the k most likely candidate tokens.

Top-p (nucleus)

Samples from the smallest candidate set whose cumulative probability reaches p.

Max output tokens

Places a limit on generated response length.

Stopping

Generation may stop at an end token, a configured stop sequence or the output-token limit.

Hallucination

Plausible-sounding but unsupported or incorrect output. Reduce risk with trusted context, verification, tools and explicit uncertainty handling.

Key takeaway: Decoding changes how the model samples tokens; it does not add factual knowledge to the model.

4. Prompt Engineering & Conversation Design

Prompt structure

State the task, provide context, define the audience, specify constraints and describe the required output.

Zero-shot prompting

Ask the model to perform a task without examples.

Few-shot prompting

Provide examples of the desired input-output pattern.

Role prompting

Define a role or perspective to guide tone and task framing.

Output constraints

Request a format such as JSON, a table, bullet points or a fixed schema.

Prompt template

Role: study assistant. Task: explain the concept. Audience: beginner CSE student. Output: definition, three key points and one example.

Message roles

System/developer instructions set behaviour; user messages contain requests; assistant messages contain responses. Tool calls and tool results can appear as separate messages.

Prompt injection

Untrusted content may contain instructions that try to override the task. Separate instructions from source content and enforce tool permissions.

Key takeaway: Clear prompts improve consistency, but prompting alone cannot guarantee correctness or safety.

5. Memory, Structured Output, Embeddings & Fine-Tuning

In-context history

Earlier messages included in the current prompt; useful for conversational continuity but consumes context tokens.

Persistent memory

Information retained between sessions, subject to the application's memory design and controls.

External knowledge

Documents or databases retrieved when needed; often used in RAG.

Structured output

Constrains responses to a schema, often using Pydantic or JSON Schema. Validate values and business rules separately.

Example schema

from pydantic import BaseModel\nclass Movie(BaseModel):\n title: str\n year: int\n rating: float

Embeddings

Dense vectors representing semantic information. Used for semantic search, clustering and retrieval.

Fine-tuning

Continues training a pretrained model on task- or domain-specific examples to adapt behaviour, style or task performance.

LoRA

Parameter-efficient fine-tuning that learns small low-rank update matrices while most base weights remain frozen.

QLoRA

Combines quantized base weights with LoRA adapters to reduce memory requirements.

Key takeaway: RAG is usually better for frequently changing facts; fine-tuning is useful for adapting behaviour and task patterns.

6. Evaluation, Applications & Interview Revision

Evaluation dimensions

Correctness, relevance, groundedness, safety, latency, cost and reliability.

Evaluation methods

Use representative test cases, task-specific metrics, human review where appropriate, and tests for edge cases and tool failures.

Application patterns

Summarization, extraction, classification, RAG, tool use and multi-step agents.

Typical system flow

User request → prompt construction → model call → optional retrieval/tool call → validation → final response.

Risks and controls

Hallucination—grounding and verification. Data exposure—minimize data and enforce access control. Prompt injection—treat retrieved content as untrusted. Cost—caching, smaller models and shorter context.

Prompting vs RAG vs fine-tuning

Prompting changes instructions; RAG supplies external context at inference time; fine-tuning changes model parameters.

Interview questions

What is next-token prediction? Explain tokenization, embeddings, attention and context window. Compare temperature, top-k and top-p. Compare RAG and fine-tuning. How do you evaluate hallucinations?

Key takeaway: Evaluate the complete application—not just the base model—because prompts, retrieval, tools and orchestration affect quality.